Here's a cleaned, de-duplicated E3 spec you can drop into the Evaluation doc and then implement against.

---

# E3 – LLM Tools & Evaluation Harness

## 1. Purpose

E3 adds a controlled LLM layer on top of Slice's numeric outputs and context, turning theses, observations, and daily state into structured, testable objects:

- No free-form chat.
- No hidden side effects (no DB, no retrieval).
- All outputs are JSON, validated against Pydantic DTOs.
- All tools are contract-tested with stubbed LLM responses.
- Metrics are captured for latency, usage, and errors.

The point is to evaluate:

- Can these tools reliably produce structured insight on top of E2?
- Can we bound hallucinations via schemas + reference validation + "insufficient_context" semantics?

---

## 2. LLM Tool Surface

We expose exactly four LLM "tools":

```
def llm_review_thesis(
    thesis: Thesis,
    evaluation: ThesisEvaluationResult,
) -> ThesisReview: ...

def llm_cross_theses(
    theses: list[Thesis],
) -> CrossThesisReport: ...
```

```
def llm_query_intuition(
    question: str,
    observations: list[Observation],
) -> IntuitionAnswer: ...

def llm_daily_summary(
    context: DailyContext,
) -> DailySummary: ...
```

**Constraints:**

- Pure functions: no DB, no network beyond the LLM call itself.
- All required context is in the arguments.
- Return types are strict Pydantic models defined in src/slice/models/llm_outputs.py.

---

## 3. DTO Schemas

### 3.1 Output models (src/slice/models/llm_outputs.py)

```
class ThesisReview(BaseModel):
    critique: str
    questions: list[str]
    risk_flags: list[str]
    insufficient_context: bool

class CrossThesisReport(BaseModel):
    overlaps: list[str]
    contradictions: list[str]
    gaps: list[str]
    insufficient_context: bool

class IntuitionAnswer(BaseModel):
    answer: str
    references: list[str]  # observation IDs used
    insufficient_context: bool

class DailySummary(BaseModel):
    key_narratives: list[str]
    risk_highlights: list[str]
    thesis_references: list[str]  # thesis IDs or titles
    insufficient_context: bool
```

Fields are **required**. Empty output is expressed as empty lists / blank strings, not missing keys.

### 3.2 Input context models

If not already defined, add:

```
class DailyContext(BaseModel):
    date: date
    portfolio_snapshot: PortfolioSnapshot  # existing or new model
    alerts: list[Alert]                    # e.g. disconfirmers, triggers
    observations: list[Observation]        # top 1–N daily obs
    active_theses: list[Thesis]            # or (id, title) metadata
```

IDs that we want the LLM to reference (Observation.id, Thesis.id) must be present in the context.

---

## 4. Prompting & JSON Handling

All four tools follow the same pattern:

1. **Build a prompt** from inputs.
2. **Run** via SessionOrchestrator.
3. **Parse** the response as JSON into the target DTO.
4. **Validate references** (where applicable).
5. **Record metrics**.
6. Return the DTO or raise an explicit error.

### 4.1 System prompt pattern

System role is always:

- Role: expert macro assistant / analyst.
- Grounding rule:

Use ONLY the information explicitly provided in the prompt.
Do not use outside knowledge.
If you cannot answer from the provided data, set "insufficient_context": true and keep your answer minimal.

- Format rule:

Respond with a single valid JSON object matching this schema and nothing else.

Each tool then defines its own schema snippet in the prompt.

### 4.2 Per-tool prompt semantics

**llm_review_thesis**

- System: "You are a macro investment analyst reviewing an investment thesis against its quantified evaluation."
- User context includes:
  - Thesis summary: title, hypothesis, drivers, disconfirmers, expression legs (assets, direction, sizing).
  - Key E2 metrics: total return, vol, Sharpe, max DD, and *summarized* scenario outcomes (no full time series).
- Model is asked to:
  - Critique logical soundness / structure.
  - Pose clarifying questions.
  - Flag risk areas.
- Output must conform to:

```
{
  "critique": "...",
  "questions": ["..."],
  "risk_flags": ["..."],
  "insufficient_context": false
}
```

**llm_cross_theses**

- System: "You analyze relationships between investment theses."
- User context: list of theses (IDs/titles + 1–2 line summary each; and high-level expression summary if useful).
- Model tasks:
  - overlaps: common themes / exposures.
  - contradictions: directly opposing assumptions or exposures.
  - gaps: missing angles, blind spots, questions none of them address.
- Output schema:

```
{
```

```
  "overlaps": ["..."],
  "contradictions": ["..."],
  "gaps": ["..."],
  "insufficient_context": false
}
```

**llm_query_intuition**

- System: "You answer the user's question using only the provided observations. If the answer is not contained in those observations, you must set insufficient_context to true."
- User context:
  - List of observations, each annotated with an ID:
    - [obs1] 2025-11-01: Fed raises rates by 0.25%...
  - Question text.
- Model tasks:
  - If possible, synthesize an answer citing observation IDs.
  - If not possible, **do not guess**; set insufficient_context = true, provide empty/very minimal answer.
- Output schema:

```
{
  "answer": "...",
  "references": ["obs1", "obs2"],
  "insufficient_context": false
}
```

**llm_daily_summary**

- System: "You write a concise daily summary for a portfolio manager, using only the provided daily context."
- User context: DailyContext flattened – portfolio moves, alerts, observations, active theses with IDs.
- Model tasks:
  - key_narratives: 2–5 bullet-level storylines about what mattered today.
  - risk_highlights: concrete risk items to watch.
  - thesis_references: IDs of impacted theses (using the IDs present in context).
- Output schema:

```
{
  "key_narratives": ["..."],
  "risk_highlights": ["..."],
  "thesis_references": ["thesis_1", "thesis_3"],
  "insufficient_context": false
}
```

### 4.3 JSON parsing and salvage

Implementation pattern (example for llm_review_thesis):

```
def llm_review_thesis(thesis: Thesis, evaluation: ThesisEvaluationResult) ->
ThesisReview:
    prompt = build_thesis_review_prompt(thesis, evaluation)
    resp = orchestrator.run_session(prompt,
options=SessionOptions(max_tokens=500, temperature=0.2))
    raw = resp.content
    latency = resp.latency_ms

    try:
        result = ThesisReview.parse_raw(raw)
    except Exception:
        json_str = extract_json(raw)  # helper: find first '{'..last '}'
        try:
            result = ThesisReview.parse_raw(json_str)
        except Exception as e:
            record_llm_call("thesis_review", latency, success=False)
            raise LLMOutputError(f"Invalid JSON from llm_review_thesis:
{raw}") from e

    record_llm_call("thesis_review", latency, success=True)
    return result
```

extract_json is a minimal helper; we are not building a full repair framework, just trimming pre/post garbage when needed.

---

## 5. Reference Validation & Hallucination Guardrails

We only validate what we can mechanistically check.

### 5.1 IntuitionAnswer references

After parsing:

```
provided_ids = {obs.id for obs in observations}
unknown = [ref for ref in result.references if ref not in provided_ids]

if unknown:
    result.references = [ref for ref in result.references if ref in
provided_ids]
    result.insufficient_context = True  # force "I relied on something I
shouldn't have"
```

We do **not** attempt semantic validation of the answer text.

### 5.2 DailySummary thesis_references

Similar to observations:

```
allowed_ids = {thesis.id for thesis in context.active_theses}
unknown = [ref for ref in result.thesis_references if ref not in allowed_ids]

if unknown:
    result.thesis_references = [ref for ref in result.thesis_references if
ref in allowed_ids]
    result.insufficient_context = True
```

### 5.3 CrossThesisReport / ThesisReview

- No hard reference fields.
- We accept free text; no automatic hallucination checks beyond schema validation. The evaluation stage is about **format + behavior**, not semantic correctness.

---

## 6. Metrics & Diagnostics

We maintain simple in-memory metrics per tool.

### 6.1 Data structure

```
class LLMToolStats(BaseModel):
    calls: int = 0
    errors: int = 0
    total_latency_ms: int = 0

    @property
    def avg_latency_ms(self) -> float:
        return self.total_latency_ms / self.calls if self.calls else 0.0


llm_stats: dict[str, LLMToolStats] = {
    "thesis_review": LLMToolStats(),
    "cross_theses": LLMToolStats(),
    "intuition_qa": LLMToolStats(),
    "daily_summary": LLMToolStats(),
}
```

### 6.2 Recording

```
def record_llm_call(tool: str, latency_ms: int, success: bool) -> None:
    stats = llm_stats[tool]
    stats.calls += 1
    stats.total_latency_ms += latency_ms
    if not success:
        stats.errors += 1
```

Each wrapper function must call record_llm_call exactly once per attempt (success or failure).

### 6.3 Diagnostics endpoint

FastAPI handler:

```
@router.get("/api/v1/diagnostics/llm")
def get_llm_diagnostics() -> dict:
    return {
        "llm_tool_metrics": {
            name: {
                "calls": s.calls,
                "errors": s.errors,
                "avg_latency_ms": s.avg_latency_ms,
            }
            for name, s in llm_stats.items()
        }
    }
```

Scope: in-memory only; multi-process aggregation is explicitly **out of scope** for E3.

---

## 7. Testing Plan (Contract Harness)

All LLM calls are stubbed in tests; no live API calls.

Use monkeypatching/fakes on SessionOrchestrator.run_session.

### 7.1 Schema contract tests

For each tool:

- Provide minimal valid inputs.
- Patch orchestrator to return a canonical JSON string.
- Assert that:
    - The function returns the correct DTO type.
    - result.dict() matches the JSON fields.
    - insufficient_context is correctly carried through.
    - record_llm_call updated metrics (calls=1, errors=0, avg_latency_ms=given value).

Example for llm_review_thesis:

- Fake JSON:

```
{
  "critique": "Plausible but over-concentrated.",
  "questions": ["What if inflation re-accelerates?"],
  "risk_flags": ["Concentration in energy"],
  "insufficient_context": false
}
```

- Check DTO fields and metrics.

Repeat analogously for:

- CrossThesisReport
- IntuitionAnswer
- DailySummary

### 7.2 Insufficient context behavior

- llm_query_intuition with **empty observations** and a non-trivial question.
- Stubbed LLM returns:

```
{"answer": "", "references": [], "insufficient_context": true}
```

- Assert result.insufficient_context is True, answer empty/placeholder, no error.

Similar for llm_daily_summary with effectively empty DailyContext.

### 7.3 Reference validation tests

- For llm_query_intuition:
    - Input observations: IDs ["obs1", "obs2"].
    - Stubbed LLM returns:

```
{
  "answer": "Growth is slowing.",
  "references": ["obs2", "obs999"],
  "insufficient_context": false
}
```

    -
    -
    - After function:
        - result.references == ["obs2"]
        - result.insufficient_context is True
- For llm_daily_summary:
    - active_theses IDs ["T1", "T2"].
    - Stubbed output uses ["T1", "T999"].
    - Expect ["T1"] and insufficient_context=True.

### 7.4 Malformed JSON robustness

Two cases per tool:

1. Valid JSON with trailing garbage:
    - Stub: '{"answer": "ok", "references": [], "insufficient_context": false} extra text'.
    - extract_json should salvage the JSON.
    - Assert parse success and metrics success.
2. Completely invalid:
    - Stub: 'Not JSON at all'.
    - Expect LLMOutputError (or ValueError) and:
        - llm_stats[tool].calls == 1
        - llm_stats[tool].errors == 1

### 7.5 Metrics correctness

- Run two successful fake calls with latency 1000 and 500 ms.

- Assert avg_latency_ms == 750.0.
- Run one failing call and assert errors increments, calls increments, and avg_latency_ms reflects all attempts, not just successes.

---

## 8. Constraints, Risks, Non-goals

### Constraints

- Only one external dependency in this layer: whatever LLM the orchestrator uses.
- No schema evolution in E3: if DTOs change, prompts + tests must be updated in lockstep.
- Observations / theses passed in must already be filtered/summarized by upstream components (we are not solving context length here).

### Risks

- Model might output semantically wrong but syntactically valid JSON. This is out of scope: E3 is about **structure + behavior**, not truth.
- Lower-quality models might ignore JSON-only instructions; then we rely on extract_json + hard failures.

### Non-goals

- No semantic scoring of whether the critique is "good."
- No automatic retries on bad JSON.
- No persistent metrics store or distributed aggregation.

---

## 9. Hard Deliverables for E3

1. **DTO module**
    - src/slice/models/llm_outputs.py with the four output models.
    - DailyContext (if missing) added to models or context module.
2. **LLM tool implementations**
    - src/slice/llm/tools.py (or equivalent) containing:
        - llm_review_thesis
        - llm_cross_theses

        - llm_query_intuition
        - llm_daily_summary
    - Shared helpers: build_*_prompt, extract_json.
3. **Metrics & diagnostics**
    - llm_stats + record_llm_call helper.
    - GET /api/v1/diagnostics/llm endpoint returning aggregated stats.
4. **Test suite**
    - Contract tests per tool for:
        - Happy-path DTO parsing.
        - Insufficient-context behavior.
        - Reference validation behavior.
        - Malformed JSON handling.
        - Metrics correctness.

Once these four pieces are in place and tests are green, E3 is functionally complete and ready to be wired into the higher-level evaluation flows.